Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Entregable 5: Documentación del sistema de correlación e inferencia
Sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Documentación del sistema de correlación e inferencia 1

- Propósito y alcance 1
- Ubicación en la arquitectura 1
- Flujo de procesamiento 3
- Entrada: el evento estandarizado 4
- Estado en Redis 5
- Reglas de correlación 6
- Inferencia: score de confianza 8
- Control de repeticiones y duplicados 9
- Detección y análisis de falsos positivos 10
- Salida: la alerta 10
- Objetivo de nivel de servicio (SLO) de latencia 10
- Validación con escenarios guionados 10
- Garantías ante fallas 12
- Rendimiento 12
- Parámetros configurables 13
- Limitaciones y mejoras futuras 13

Referencias 14

Documentación del sistema de correlación e inferencia

Propósito y alcance

Este documento describe el sistema de correlación e inferencia del monitoreo perimetral urbano de San Juan Sacatepéquez: cómo recibe los eventos de los sensores, qué estado mantiene, qué reglas aplica para inferir incidentes, cómo calcula la confianza de cada alerta, cómo detecta los falsos positivos y qué garantías ofrece ante fallas. Corresponde al quinto entregable de la sección 8 del enunciado. Está escrito para que el equipo pueda operarlo, explicarlo y extenderlo, y cita los archivos del repositorio donde vive cada parte (carpeta `servicios/correlador`).

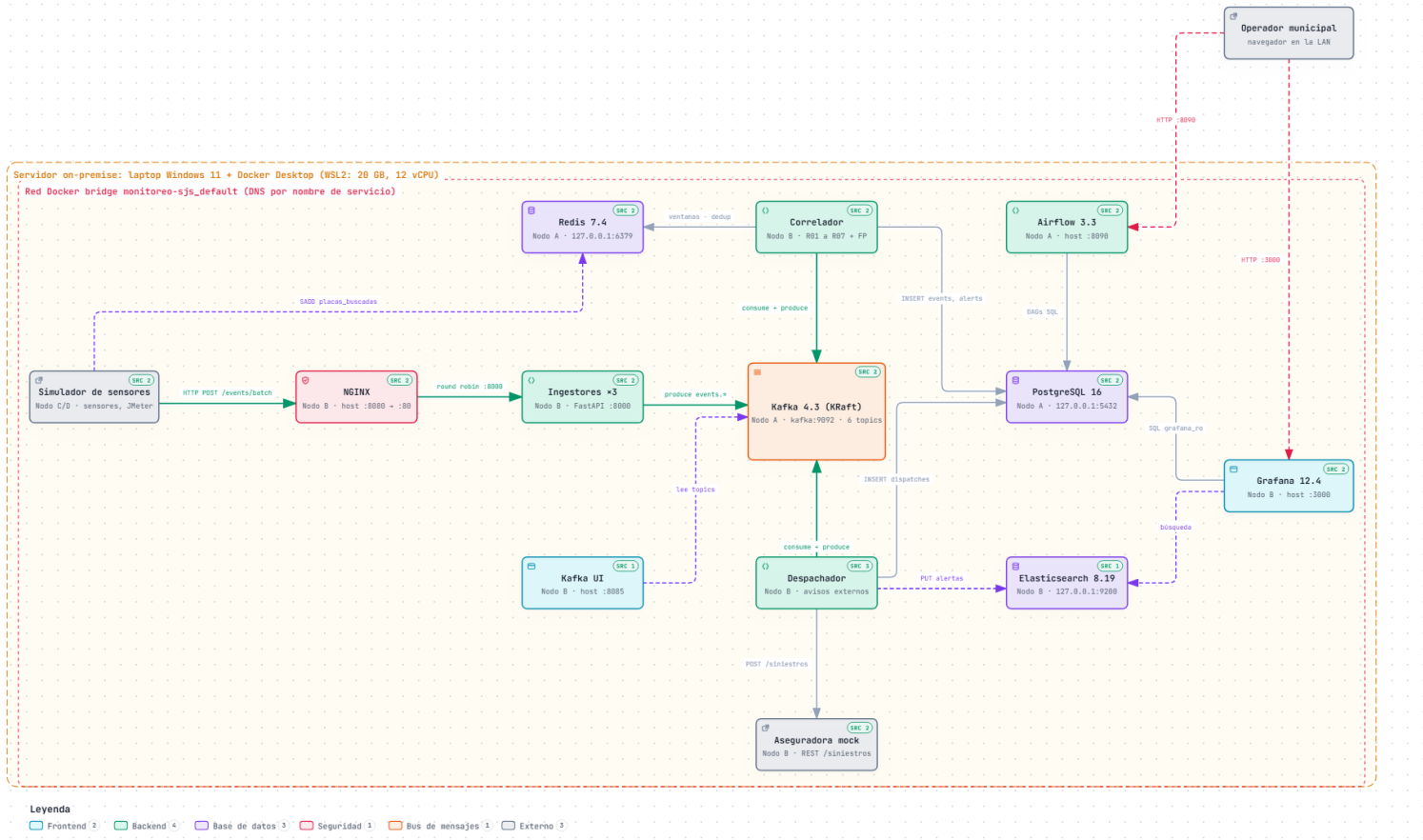
La correlación de eventos consiste en relacionar sucesos de fuentes distintas que ocurren cerca en el tiempo y en el espacio para inferir algo que ninguna fuente sola permite afirmar (Luckham, 2002). Por ejemplo, un botón de pánico por sí solo puede ser un error, pero un pánico seguido de un vehículo que sale del mismo sector con exceso de velocidad indica un posible robo en curso, que es justamente el ejemplo de la sección 2.4 del enunciado.

Ubicación en la arquitectura

El correlador es un consumidor de Kafka (grupo `correlador`) que lee el topic `events.standardized`, donde los ingestores publican los eventos ya validados con la clave igual al sector. Por cada evento consulta y actualiza su estado en Redis, guarda el evento y las alertas en PostgreSQL y publica cada alerta en el topic `correlated.alerts`, desde donde el despachador la enruta a la PNC, Bomberos, CONRED o la aseguradora. Es decir, el correlador es consumidor y productor a la vez.

Figura 1

Ubicación del correlador en la infraestructura del sistema



Nota. Las flechas indican quién abre la conexión (cliente hacia servidor). Diagrama generado a partir del código del repositorio; versión interactiva en diagrama-infraestructura.html.

Tabla 1

Archivos del correlador

Archivo	Responsabilidad
correlador.py	Bucle principal: consume lotes de Kafka, procesa cada evento, revisa los pánicos pendientes cada 10 s y confirma el lote
reglas.py	Una función por regla (R01 a R07) y la regla de falso positivo
calculos.py	Distancia entre coordenadas (haversine), horario escolar y score de confianza
estado_redis.py	Todo lo que se guarda y consulta en Redis
base_datos.py	Escritura de eventos, alertas y violaciones de SLO en PostgreSQL

Flujo de procesamiento

El correlador implementa los ocho pasos del diagrama de referencia del catedrático. La Tabla 2 relaciona cada paso con lo que hace el código.

Tabla 2

Pasos del correlador según el diagrama de referencia

Paso	Qué hace el sistema
1. Recibo evento desde Kafka	Lee lotes de hasta 500 mensajes de events.standardized con espera máxima de 0.2 s y descarta duplicados por eventId
2. Busco el vehículo o el sector en Redis	Pide los eventos recientes del sector y, si el evento es de una cámara LPR, el último paso registrado de esa placa
3. Redis responde con el último estado	Devuelve la ventana del sector (los 20 eventos más recientes de cada tipo dentro de la ventana) y el último sensor, sector y hora del vehículo
4. Correlaciono A + B	Combina el evento nuevo con los anteriores del mismo sector que no se hayan usado en una alerta del mismo tipo
5. Calculo velocidad (si aplica)	Para la regla R05, distancia entre cámaras con la fórmula de haversine dividida entre el tiempo transcurrido
6. Aplico regla de negocio	Evalúa las reglas R01 a R07; una regla solo se dispara si el evento nuevo participa en ella
7. Guardo nuevo estado en Redis	Agrega el evento a la ventana, registra el pánico como pendiente y actualiza el último paso del vehículo
8. Persisto el incidente y publico la alerta	Inserta el evento y la alerta en PostgreSQL y publica la alerta en correlated.alerts

Al terminar cada lote, el orden de confirmación es fijo: primero se confirma la transacción de PostgreSQL, luego se espera la confirmación de Kafka para las alertas publicadas, después se marcan los eventos como procesados en Redis y al final se confirma el offset en Kafka. Este orden garantiza la entrega al menos una vez: si el proceso muere en cualquier punto, el lote se vuelve a leer y lo que ya se había hecho se reconoce y se omite.

Entrada: el evento estandarizado

Cada evento sigue un sobre común basado en el ejemplo del catedrático, con el campo `schemaVersion`, un identificador único (`eventId`), el tipo de evento, la hora en que ocurrió (`timestamp`) y la hora en que se envió (`emittedAt`), además de los bloques `device`, `location`, `data` y `metadata`. El ingestor agrega el bloque `ingestion` con la hora de recepción y el ingestor que lo atendió. El contenido de `data` depende del sensor (Tabla 3).

Tabla 3

Datos de cada tipo de sensor

Sensor	eventType	Campos de data usados por las reglas
Botón de pánico	security.panic_button.pressed	alertType (PANICO, EMERGENCIA_MEDICA, INCENDIO), origin (BOTON_FIJO, APP_MOVIL)
Cámara LPR	traffic.lpr.read	plateHash (HMAC-SHA256), platePartial, estimatedSpeedKmh, direction (ENTRADA, SALIDA)
Radar de velocidad	traffic.speed.reading	speedKmh, limitKmh, anomalyType (NINGUNA, EXCESO_VELOCIDAD, TRAYECTORIA_ANOMALA)
Sensor acústico	environment.acoustic.detected	soundType (DISPARO, EXPLOSION, VIDRIO_ROTO), criticalProbability
Reporte ciudadano	citizen.report.created	reportType (ACCIDENTE, INCENDIO, ALTERCADO), message (texto libre), accuracyMeters

El campo `scenarioId` solo lo traen los escenarios de prueba. El correlador nunca lo usa para decidir; únicamente lo copia a la alerta cuando todos los eventos de origen

pertenecen al mismo escenario, para que la herramienta de evaluación pueda medir aciertos y falsos positivos.

Estado en Redis

Redis es la memoria de corto plazo del correlador. Todo el estado necesario para correlacionar vive ahí y no dentro del proceso, lo que permite reiniciar el correlador sin perder el contexto y correr varias copias en paralelo. Redis guarda su contenido en disco con el modo AOF (append only file).

Tabla 4

Claves de Redis usadas por el correlador

Clave	Tipo	Contenido y vigencia
evento:{eventId}	texto	Marca de evento ya procesado (descarte de duplicados); 1 hora
ventana:{sector}:{tipo}	sorted set	Eventos recientes relevantes del sector, ordenados por la hora del evento; se recorta a la ventana y expira a las dos ventanas
vehiculo:{plateHash}	texto (JSON)	Último paso del vehículo: cámara, sector, hora y coordenadas; 1 hora
panicos_pendientes	sorted set	Pánicos que todavía no tienen confirmación, por hora del evento
panico:{eventId}	texto (JSON)	Resumen del pánico pendiente, para generar el falso positivo
usado:{tipoAlerta}:{eventId}	texto	Evento ya usado en una alerta de ese tipo; dos ventanas
cooldown:{regla}:{sujeto}	texto	Bloquea repetir la misma alerta para el mismo sector o placa durante una ventana
placas_buscadas	set	Hashes de las placas reportadas como robadas (los carga cargar_buscados.py)

Ventana de tiempo

La ventana de correlación (`CORRELATION_WINDOW_SECONDS`) es de 300 s en operación normal y de 60 s en la demostración. Se mide con la hora en que ocurrió el evento y no con el reloj del servidor: así, cuando el correlador procesa un atraso después de una caída, los eventos de ese atraso se siguen correlacionando entre sí como si hubieran llegado a tiempo.

Solo entran a la ventana los eventos que alguna regla busca en ella: pánicos, sensores acústicos, reportes, lecturas de velocidad con anomalía y lecturas LPR de salida. Las lecturas normales de velocidad y las entradas LPR no se guardan, porque las reglas que las usan (R05, R06 y R07) trabajan solo con el evento nuevo, el vehículo o la lista de placas buscadas. Además, de cada tipo se leen como máximo los 20 eventos más recientes. Esta decisión subió la capacidad del correlador de 66 a 556 eventos por segundo, porque antes cada evento leía y decodificaba miles de eventos de la ventana bajo carga.

Reglas de correlación

Las reglas son determinísticas: dadas las mismas entradas, producen siempre la misma salida, y cada alerta indica qué regla la generó y cuáles fueron sus eventos de origen (`sourceEventIds`), lo que permite auditarla hasta el sensor. Salvo R05, todas exigen el mismo sector y que los eventos estén dentro de la ventana.

Tabla 5

Reglas de correlación del sistema

Regla	Alerta	Condición	Severidad
R01	ROBO_EN_CURSO	Pánico (PANICO) + lectura LPR con dirección SALIDA + anomalía de velocidad (radar con EXCESO_VELOCIDAD o la misma cámara sobre el límite del sector)	CRITICA
R02	ATAQUE_ARMADO	Sonido DISPARO con probabilidad mayor o igual a 0.7 + pánico	CRITICA
R03	ACCIDENTE_VIAL	Reporte ACCIDENTE + radar con TRAYECTORIA_ANOMALA o EXCESO_VELOCIDAD; o bien dos reportes ACCIDENTE	ALTA
R04	INCENDIO_CONFIRMADO	Pánico de tipo INCENDIO + reporte INCENDIO o sonido EXPLOSION	CRITICA
R05	VELOCIDAD_ENTRE_CAMARAS	Misma placa (mismo hash) en dos cámaras distintas en menos de una hora; velocidad promedio del tramo mayor a 80 km/h	MEDIA (ALTA si alguna cámara está en zona escolar en horario escolar)
R06	ZONA_ESCOLAR	Lectura de radar mayor a 30 km/h en un sector escolar dentro del horario escolar	MEDIA
R07	VEHICULO_REPORTADO	El hash de la placa está en el conjunto placas_buscadas	ALTA
FP	PANICO_NO_CONFIRMADO	Pánico que no participó en ninguna alerta al cerrarse su ventana	BAJA (estado FALSO_POSITIVO_PROBABLE)

Selección de eventos

Cuando varias combinaciones cumplen una regla, se usa el evento más reciente de cada tipo que todavía no se haya usado en una alerta del mismo tipo. En R01 se prefiere una lectura LPR de salida que ya venga sobre el límite; si no la hay, se exige

además un radar con exceso de velocidad. Una regla solo se dispara si el evento recién llegado participa en ella, para que una combinación vieja no vuelva a generar la alerta con cada evento que llega.

Cálculo de velocidad entre cámaras

La regla R05 calcula la distancia entre las coordenadas de las dos cámaras con la fórmula de haversine (Sinnott, 1984), que da la distancia sobre la superficie de la Tierra a partir de latitud y longitud, y la divide entre el tiempo transcurrido entre las dos lecturas. En el escenario de prueba, una placa pasa por una cámara en Lo de Mejía y 120 s después por otra en Sajcavillá: 3.33 km en 120 s equivalen a 100 km/h, por encima del umbral de 80 km/h. La lectura de un vehículo solo se guarda si es más reciente que la registrada, porque al procesar un atraso de particiones distintas pueden llegar antes lecturas nuevas.

Horario escolar

El horario escolar se configura con `SCHOOL_HOURS` (por defecto de 06:30 a 08:00 y de 12:00 a 13:30) y se evalúa con la hora del evento en Guatemala (UTC-6, sin horario de verano). Los sectores escolares son Casco urbano, Cruz Blanca y Ciudad Quetzal.

Inferencia: score de confianza

Cada alerta incluye un score entre 0 y 1 que resume qué tan confiable es la inferencia. Se calcula como la proporción de condiciones cumplidas multiplicada por el promedio de la confianza de los sensores que participaron:

score = (condiciones cumplidas / condiciones de la regla) × promedio de la confianza de los eventos de origen

Tabla 6

Confianza asignada a cada tipo de sensor

Sensor	Confianza	Razón
Acústico	Su criticalProbability	El propio sensor estima la probabilidad de un evento crítico
Botón de pánico fijo	0.90	Dispositivo instalado y controlado por la municipalidad
Pánico desde la app	0.80	Más expuesto a toques accidentales
Reporte ciudadano	0.80 si la precisión es de 50 m o menos; 0.70 en otro caso	La ubicación aproximada reduce la certeza
Cámara LPR y radar	0.95	Medición automática de alta precisión

Como las alertas se generan cuando todas las condiciones se cumplen, el primer factor vale 1, salvo en el falso positivo, donde vale 1/2 (hubo pánico, pero nada lo confirmó). Por ejemplo, la alerta de ataque armado del escenario de prueba combina un disparo con probabilidad 0.9 y un pánico: si el pánico viene de un botón fijo (0.90), el score es 0.90; si viene de la app (0.80), es 0.85. Un falso positivo de un pánico de botón fijo tiene score 0.45.

Control de repeticiones y duplicados

- **Duplicados de entrada:** si llega dos veces el mismo eventId (por un reintento del cliente o de NGINX), se descarta; además, la tabla de eventos usa ON CONFLICT DO NOTHING.
- **Eventos usados:** un evento que participó en una alerta de un tipo no se reutiliza para otra alerta del mismo tipo.
- **Cooldown:** cuando una regla se dispara, no vuelve a dispararse para el mismo sujeto durante una ventana. El sujeto es el sector en R01 a R04 y R06, y la placa en R05 y R07, porque dos vehículos distintos en el mismo sector son dos alertas distintas.
- **Atomicidad:** el cooldown se toma con SET NX y el falso positivo se retira de la lista con ZREM, operaciones atómicas de Redis que impiden que dos copias del correlador generen la misma alerta.

Detección y análisis de falsos positivos

Cada pánico entra a la lista `panicos_pendientes`. Si participa en una alerta (robo, ataque armado o incendio), sale de la lista porque quedó confirmado. Cada 10 s, el correlador revisa los pánicos cuya ventana ya se cerró y genera para cada uno una alerta `PANICO_NO_CONFIRMADO` con estado `FALSO_POSITIVO_PROBABLE` y severidad baja. Estas alertas no se envían a ninguna institución: solo se registran.

El escenario ESC-FALSO-01 (un pánico aislado en Montúfar) reproduce el segundo ejemplo del enunciado, y ESC-RUIDO-01 demuestra que un vidrio roto con probabilidad baja (0.4) no basta para una alerta crítica. Además, el proceso periódico `analisis_falsos_positivos` de Airflow calcula cada hora, por sector, la proporción de pánicos no confirmados frente a los confirmados en las últimas 24 horas, y el dashboard 3 de Grafana la muestra. Esto permite detectar sectores con botones defectuosos o mal uso de la app.

Salida: la alerta

Cada alerta se guarda en la tabla `alerts` y se publica en `correlated.alerts` con la clave igual al sector. Sus campos son: identificador, tipo, severidad, estado, sector y nombre del sector, coordenadas, score, regla, descripción en lenguaje natural, eventos de origen, escenario (si aplica), hora de creación y latencia en milisegundos.

Objetivo de nivel de servicio (SLO) de latencia

La latencia de una alerta es la hora de creación menos el `emittedAt` del último evento que completó la regla. El objetivo es menos de 2 000 ms (`SLO_LATENCIA_MS`). Si se supera, el correlador escribe una advertencia en el log y registra la alerta en la tabla `slo_violations`, que el dashboard 2 de Grafana muestra en rojo. Los falsos positivos no cuentan, porque por diseño esperan la ventana completa; su latencia se mide desde el cierre de la ventana.

Validación con escenarios guionados

El simulador incluye nueve escenarios con su alerta esperada. El script `evaluar.py` compara las alertas esperadas con las obtenidas (por scenarioId), guarda el resultado en la tabla `evaluation_results` y muestra los aciertos, los faltantes y las

alertas adicionales. La Tabla 7 muestra la evaluación de la versión final, con ventana de 60 s y los escenarios acelerados cuatro veces.

Tabla 7

Resultado de la evaluación de escenarios

Escenario	Secuencia simulada	Alerta esperada	Resultado	Latencia
ESC-ROBO-01	Pánico; LPR de salida a 85 km/h; radar a 92 km/h en Ciudad Quetzal	ROBO_EN_CURSO	Acierto	140 ms
ESC-ARMADO-01	Disparo con probabilidad 0.9; pánico en Casco urbano	ATAQUE_ARMADO	Acierto	30 ms
ESC-ACCIDENTE-01	Trayectoria anómala; reporte de accidente en Lo de Mejía	ACCIDENTE_VIAL	Acierto	158 ms
ESC-INCENDIO-01	Pánico de incendio; reporte de incendio en Villas del Quetzal	INCENDIO_CONFIRMADO	Acierto	98 ms
ESC-TRAMO-01	Misma placa en Lo de Mejía y Sajcavillá a unos 100 km/h	VELOCIDAD_ENTRE_CAMARAS	Acierto	185 ms
ESC-ESCOLAR-01	Radar a 45 km/h en Casco urbano en horario escolar	ZONA_ESCOLAR	Acierto	115 ms
ESC-BUSCADO-01	LPR de una placa reportada como robada en Cruz Blanca	VEHICULO_REPORTADO	Acierto	110 ms
ESC-FALSO-01	Un pánico aislado en Montúfar	PANICO_NO_CONFIRMADO	Acierto	8.9 s*
ESC-RUIDO-01	Pánico y vidrio roto con probabilidad 0.4 en Sajcavillá	PANICO_NO_CONFIRMADO	Acierto	8.9 s*

Nota. Resultado: 9 de 9 aciertos. *La latencia de los falsos positivos se mide desde el cierre de la ventana y depende de la revisión cada 10 s; no cuenta para el SLO. ESC-ROBO-01 genera además una alerta ZONA_ESCOLAR correcta, porque el radar de 92 km/h está en un sector escolar.

Durante la validación se identificaron tres comportamientos que conviene conocer al operar el sistema. Primero, con una ventana de 60 s el escenario de accidente queda en el límite, porque sus eventos están separados exactamente 60 s; en la demostración se ejecuta acelerado. Segundo, inmediatamente después de una prueba de carga muy alta, los cooldowns activados por correlaciones al azar bloquean durante una ventana las alertas de los escenarios en esos sectores. Tercero, la regla

R01 es sensible al ruido: en tráfico normal casi siempre hay algún vehículo saliendo, por lo que en la práctica basta un pánico y un exceso de velocidad en el mismo sector; por eso el simulador mantiene raras las anomalías del tráfico normal.

Garantías ante fallas

Tabla 8

Comportamiento del correlador ante fallas (medido)

Situación	Comportamiento	Resultado medido
Reinicio del correlador a mitad de tráfico	Atiende SIGTERM, termina el lote, cierra el consumidor y Kafka reasigna las particiones de inmediato	448 aceptados = 448 guardados; 0 duplicados; de vuelta en menos de 1 s
Correlador detenido con eventos en cola	Kafka conserva los eventos; al volver los procesa a 660 eventos/s	15 012 eventos procesados, 0 perdidos
Reinicio de Redis	El cliente se reconecta; Redis recupera su estado del AOF	0 perdidos; latencia máxima de 3.2 s durante el reinicio
Falla de PostgreSQL o Redis	El proceso termina y Docker lo reinicia; retoma desde el último offset confirmado	Entrega al menos una vez

Rendimiento

Tabla 9

Capacidad del correlador

Versión o configuración	Eventos por segundo
Primera versión (leía toda la ventana del sector con cada evento)	66
Ventana por tipo de evento, solo eventos relevantes, máximo 20 por tipo	556
Lo anterior más inserción en PostgreSQL por lote	660
Tres correladores en paralelo (uno por partición)	743
Máximo sostenido de extremo a extremo cumpliendo el SLO de 2 s	unos 510

Nota. Con tres correladores la mejora es limitada porque la partición más cargada concentró el 60 % de los eventos: la clave de partición es el sector y los sectores tienen tráfico distinto.

Parámetros configurables

Tabla 10

Variables de entorno del correlador

Variable	Valor por defecto	Efecto
CORRELATION_WINDOW_SECONDS	300 (60 en la demo)	Duración de la ventana de correlación y del cooldown
ACOUSTIC_MIN_PROBABILITY	0.7	Probabilidad mínima de disparo para la regla R02
SLO_LATENCIA_MS	2000	Umbral de latencia de alertas
SCHOOL_HOURS	06:30-08:00,12:00-13:30	Horario escolar para R05 y R06

Limitaciones y mejoras futuras

- Exigir en R01 que la anomalía de velocidad sea del mismo vehículo que salió (misma placa), para reducir las correlaciones por azar.
- Usar más particiones o una clave de partición más repartida para escalar el correlador horizontalmente, aceptando el costo de perder el orden estricto por sector.
- Ajustar la confianza de cada sensor con datos reales de la operación, por ejemplo con la proporción de falsos positivos por sector que ya calcula Airflow.
- Agregar reglas nuevas es sencillo: una función en reglas.py que reciba el evento nuevo y la ventana y devuelva la alerta.

Referencias

Luckham, D. C. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.